INM376 / IN3005 Computer Graphics using OpenGL
Lab 3

In this lab, we will be working with the coursework template, which is a basic OpenGL program that incorporates several simple components that will likely be useful for your coursework. We'll experiment a bit with the template, including transformations and the camera, and then move on to making a Catmull-Rom spline, move the camera on a spline-based path, and render the spline using a VAO / VBO.

Download the demo

1. Download the zip file "OpenGLTemplate (Template2026.zip)" from the "Coursework" area on Moodle.
2. Copy the file into a suitable location for source code on your hard drive.
3. Unzip the zip file.

Open the "OpenGLTemplate.sln" file, and press 'F5' to compile the program and run it. Hopefully you see a scene consisting of a background, a planar terrain, a sphere, barrel, and a horse.

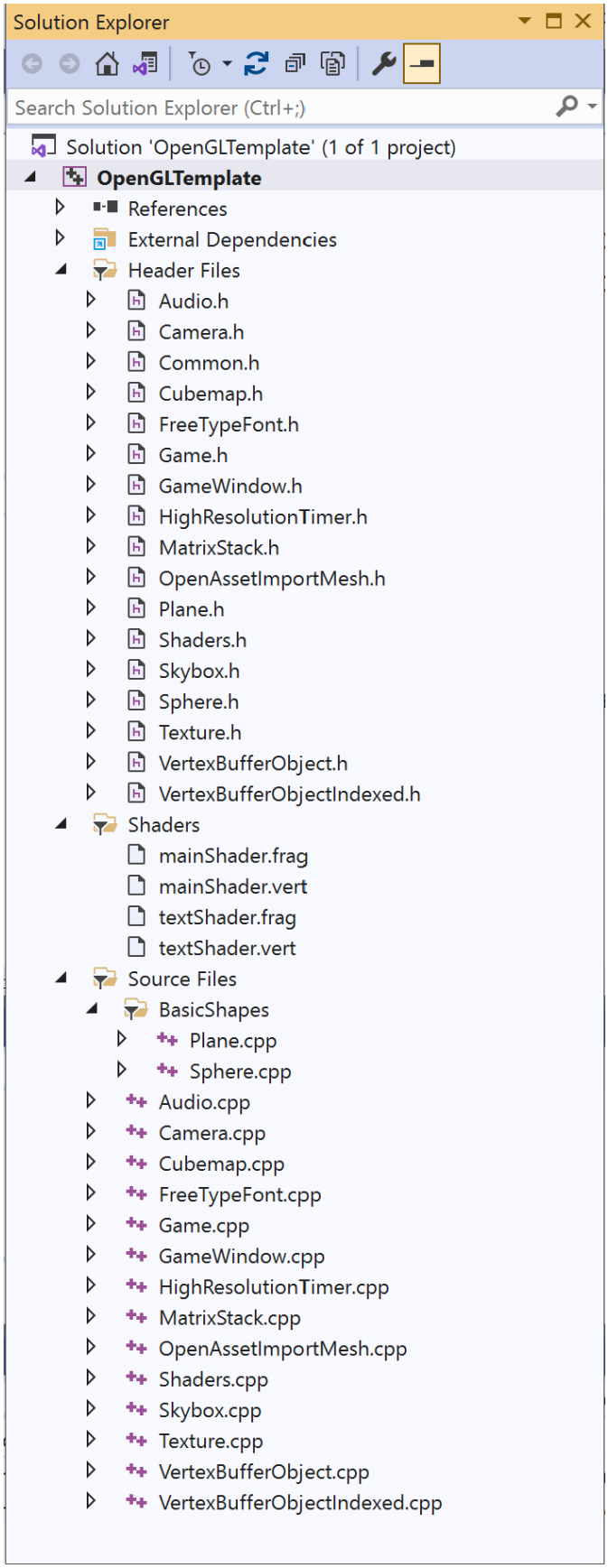
There are several features to explore. First, try moving the camera around. The mouse sets the camera orientation through rotations. The up and down arrow keys move the camera forward and backward, while the left and right arrow keys move the camera left to right (side-to-side motion like this is known as *strafing*). WASD controls are programmed as well. Take a look at the different objects in the scene.

Task 1: Exploring the OpenGL template code

The OpenGL template is written in C++ and compiles using Microsoft Visual Studio 2019. It is a program that uses WGL to create an OpenGL rendering context using OpenGL 4.0. It additionally includes a few basic shaders written in GLSL 4.0.

Take some time to explore the various parts of the template code and see how the basic graphics engine is structured. This template is provided to aid you in the development of your OpenGL graphics coursework.

In the Visual Studio Solution Explorer you will see a list of source and header files that form the template:


Solution Explorer
Search Solution Explorer (Ctrl+;)
Solution 'OpenGLTemplate' (1 of 1 project)
OpenGLTemplate
References
External Dependencies
Header Files
Audio.h
Camera.h
Common.h
Cubemap.h
FreeTypeFont.h
Game.h
GameWindow.h
HighResolutionTimer.h
MatrixStack.h
OpenAssetImportMesh.h
Plane.h
Shaders.h
Skybox.h
Sphere.h
Texture.h
VertexBufferObject.h
VertexBufferObjectIndexed.h
Shaders
mainShader.frag
mainShader.vert
textShader.frag
textShader.vert
Source Files
BasicShapes
Plane.cpp
Sphere.cpp
Audio.cpp
Camera.cpp
Cubemap.cpp
FreeTypeFont.cpp
Game.cpp
GameWindow.cpp
HighResolutionTimer.cpp
MatrixStack.cpp
OpenAssetImportMesh.cpp
Shaders.cpp
Skybox.cpp
Texture.cpp
VertexBufferObject.cpp
VertexBufferObjectIndexed.cpp

There are several features to explore in the template:

- A *skybox* to model the background
- A textured plane, forming the ground
- Camera methods
- Basic lighting methods
- An image loader that reads various formats including BMP, JPG
- A text renderer
- A high resolution timer
- Vertex and fragment shaders (in ./resources/shaders/)
- A basic mesh loader (surfaces in ./resources/models/)

Familiarise yourself with the code in the Game.cpp file and explore the GameWindow.cpp code as well. This will enable you see where the Windows and WGL code are used in the system (e.g. the keyboard controls, the main event loop). Two very important methods are Game::Initialise() and Game::Render().

Game::Initialise() does initialisation of the scene. It creates a set of objects, and sets the projection matrices. There is a projection matrix for perspective projection (used to render the 3D scene), and a projection matrix for orthographic projection (used to render 2D text). Game:: Initialise() loads a set of shaders, which are text files on the disk, compiles, links, and attaches the shaders to shader programs. In the template, vertex shaders have .vert file extensions, and fragment shaders have .frag file extensions. Game::Initialise() then creates objects like the skybox, terrain, fonts, and loads 3D meshes.

Game::Render() renders the scene. First, it clears the buffers. There is a colour buffer where each pixel has a red, green, blue, and alpha (RGBA) value, and there is also a depth buffer, which represents distance of a pixel to the camera. The depth buffer is used for visibility (we'll cover this in a later lecture). A modelview matrix stack is formed for transformations of objects. A call to LookAt sets the view matrix, based on the camera's position, view point, and up vector. A directional light source is set up and passed to the shaders using uniform variables (we'll cover this next week). Then, the scene objects are rendered. During rendering, the modelview matrix must be passed to the shader, along with the normal matrix, which is used in lighting calculations. The framerate is also rendered as text. Then, the call to SwapBuffers displays the rendered image.

Please note that in the OpenGL template, the y axis corresponds to height, or "gravity". The ground is on the xz plane.

Task 2: Positioning the camera (Part 1)

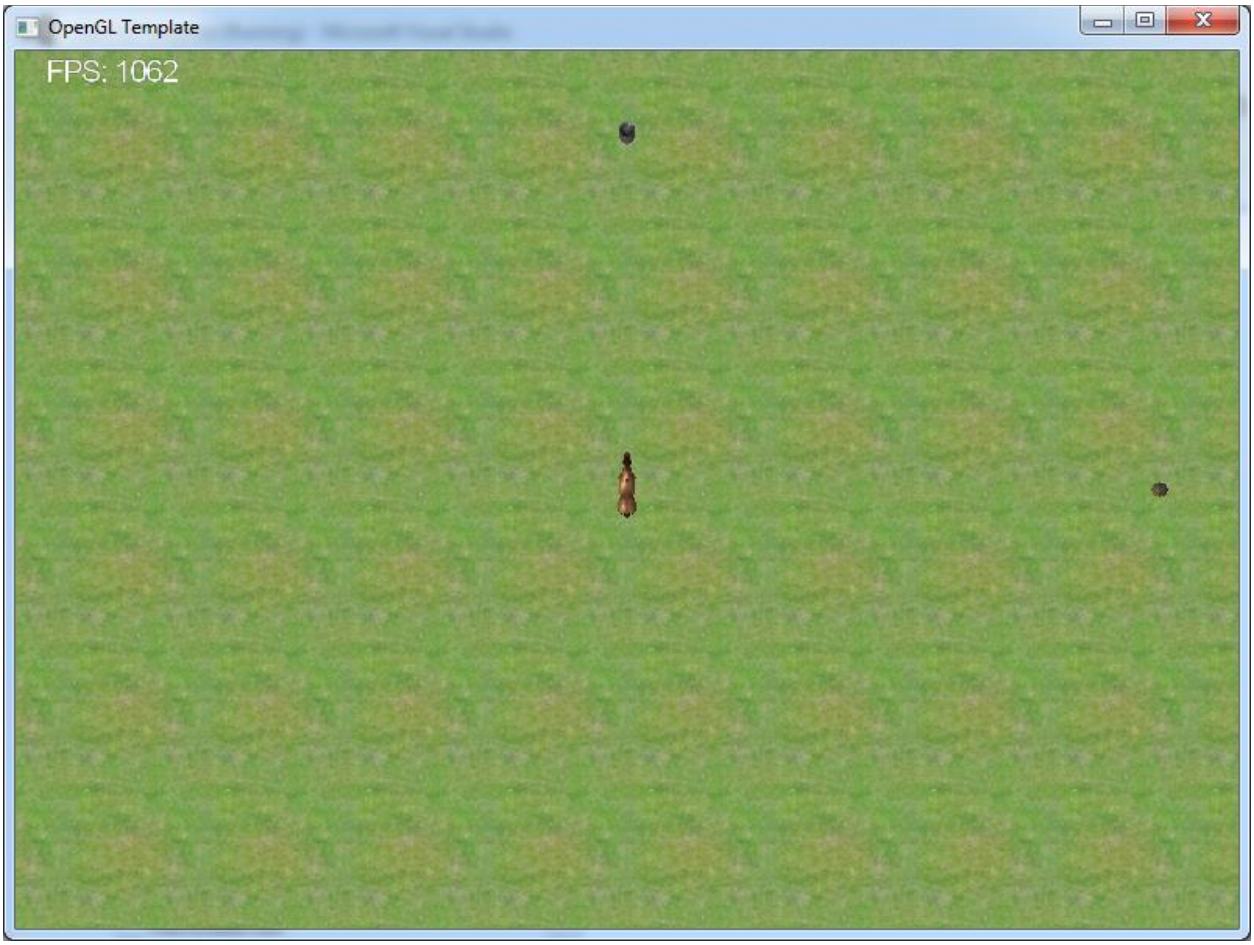
In the camera class (CCamera) constructor, you'll notice that the camera is initially set a point $[0, 10, 100]^T$. This places the camera at a point that is 10 units above the ground (where y = 0), and out 100 units along the z axis. The camera is looking at the origin $[0, 0, 0]^T$ along the negative z axis, and the up vector is the y axis. The horse is at the origin so the initial view (if you don't touch mouse at all!) will be looking towards the horse.

Change the default camera so that it is looking at the point $[100, 0, 0]^T$ instead, and rerun the program. If you are successful, the initial view (without touching the mouse) should be looking at the barrel.

Changing the camera on initialisation has its use, but typically it's more useful to set the camera's parameters on the fly. For this, there is a function

```
void CCamera::Set(glm::vec3 &position, glm::vec3 &viewpoint, glm::vec3 &upVector)
```

that allows the camera's position, viewpoint, and upvector to be set. In Game::Update(), call `m_pCamera->Set()` to set the camera so it is positioned at a point 300 units above the origin, looking at the origin, and has an upvector along the x-axis. If you do this, you should see something like the figure below.



Now comment out your call to `m_pCamera->Set()`so the camera is back to a 'free view' mode.

Task 3: Transformations

In the last lecture, we covered transformations. Attempt to resize and reposition the models in the scene (barrel, horse, sphere) by altering the modelview matrix.

Create a series of three barrels drawn in a row, by copying the code to draw the barrel and changing the modelview matrix. Add more horses to the scene, and position them at different locations and directions. Make the sphere three times larger in each dimension, but with its base still touching the ground. (Note: in local coordinates, the sphere is a unit sphere, that is centred at the origin and with a radius of one unit).

Task 4: Catmull-Rom spline interpolation

Now let's make a new class to perform Catmull-Rom interpolation. In the Solution Explorer, do a right click on Header Files and select Add->New Item, Select Header File (.h), and add a new file

CatmullRom.h. Next, in the Solution Explorer, do a right click on Source Files, select Add->New Item, Select C++ File (.cpp), and add a new file CatmullRom.cpp.

In your CatmullRom.h file, copy and paste the following code to define the class:

```
#pragma once
#include "Common.h"
#include "VertexBufferObject.h"

class CCatmullRom
{
public:
	CCatmullRom();
	~CCatmullRom();
	glm::vec3 Interpolate(glm::vec3 &p0, glm::vec3 &p1, glm::vec3 &p2, glm::vec3 &p3,
	float t);
};
```

We're going to use this class to interpolate points along a Catmull-Rom spline. Now in your CatmullRom.cpp file, include CatmullRom.h, and provide empty constructors and destructors for the class. Then, implement the Interpolate() method, based on the equations provided in today's lecture, so that it interpolates the region of space between $\mathbf{p}_1$ and $\mathbf{p}_2$. Recall the equations compute the vector constants **a**, **b**, **c**, and **d** from the points passed in, and then provides an interpolated point based on the equation $\mathbf{x}(t) = \mathbf{a} + \mathbf{b}t + \mathbf{c}t^2 + \mathbf{d}t^3$.

When the code compiles, continue to the next task.

Task 5: Fly-by, or positioning the camera (Part 2)

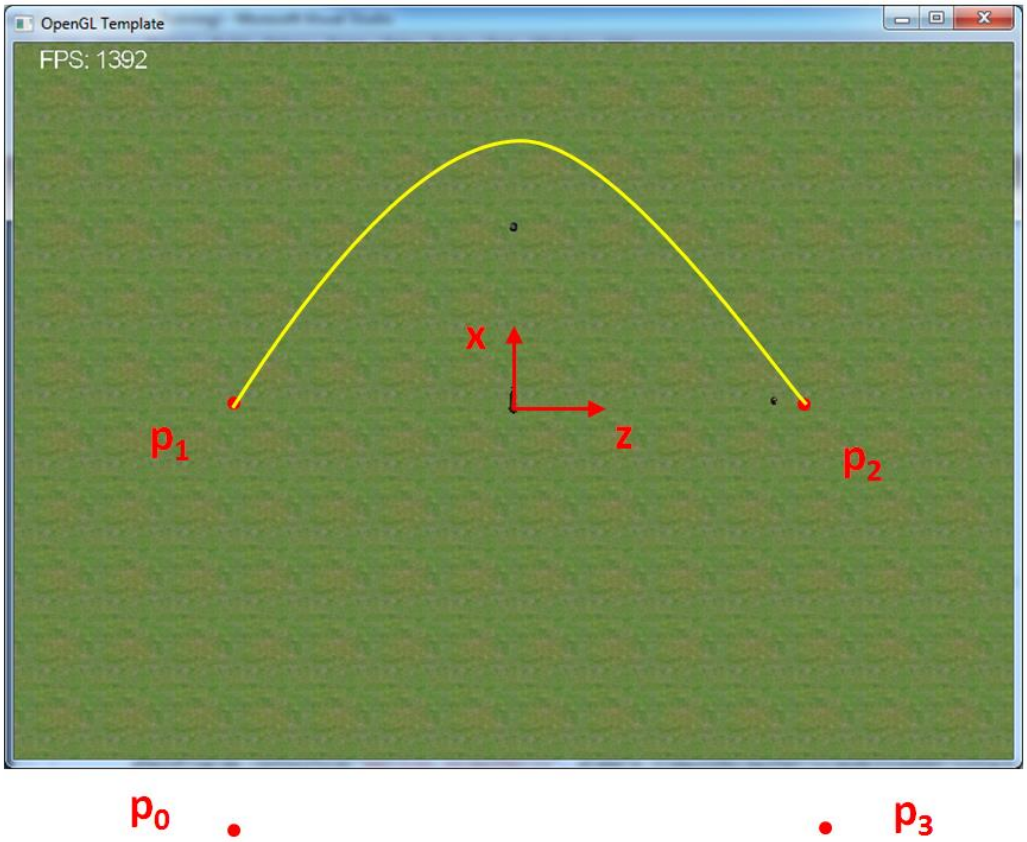
Now let's place the camera automatically using the Catmull-Rom interpolation code we just implemented. Let's use the spline to interpolate using the following four points:

$p0 = [-500, 10, -200]^T$
$p1 = [0, 10, -200]^T$
$p2 = [0, 10, 200]^T$
$p3 = [-500, 10, 200]^T$

i.e.,

```
glm::vec3 p0 = glm::vec3(-500, 10, -200);
glm::vec3 p1 = glm::vec3(0, 10, -200);
glm::vec3 p2 = glm::vec3(0, 10, 200);
glm::vec3 p3 = glm::vec3(-500, 10, 200);
```

as indicated in the figure below (not necessarily drawn to scale). The Catmull-Rom spline will interpolate between p1 and p2, as indicated with the yellow path.



In the method Game::Update(), there is a line of code to code to update the camera, based on the amount of time that has elapsed since the last frame was rendered. Comment this line out, and add the following lines of code:

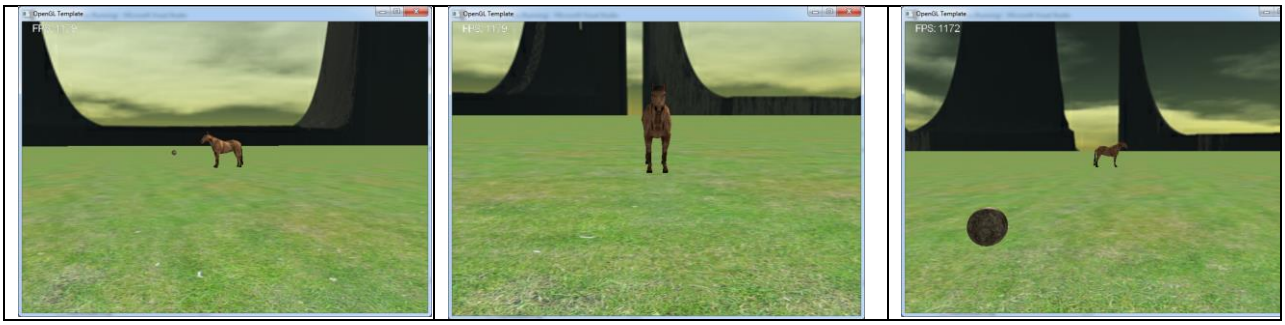
```
static float t = 0.0f;
t += 0.0005f * (float) m_dt;
if (t > 1.0f)
       t = 0.0f;
```

This code introduces a static floating point variable t, and initialises it to zero. Recall a static variable will retain its value. The variable t is being incremented by a value `0.0005f` on each frame. This is multiplied by m_dt, which is the amount of time that has elapsed since the last frame was rendered. Multiplying anything moving (such as the camera) by m_dt ensures that any motion is computed consistently, regardless of the speed of the computer that the program is run on.

Now use your Catmull-Rom spline to interpolate the camera position using the four points given above, as well as the value of t computed based on the code above. Set the camera view point so that is looking at the horse at the origin, and the camera up vector along the positive y axis. Note that in

Game.cpp it will be necessary to include “CatmullRom.h”, and instantiate a Catmull-Rom object to do the interpolation.

Run your program. You should see an animation loop, with the camera flying around the scene on a smooth non-linear path, always looking at the horse(s)! If your view is sideways for some reason, think about the upvector.



Task 6: Rendering the path using a VAO/VBO

In this task, we will sample the spline to create 100 points, and then render them using a GL_LINE_STRIP primitive. First, in Game::Update, comment out the animation of the camera, and restore the original line of code to

```
pCamera->Update(m_dt);
```

Now, in CatmullRom.h, change your class definition to this:

```
class CCatmullRom
{
public:
	CCatmullRom();
	~CCatmullRom();
	glm::vec3 Interpolate(glm::vec3 &p0, glm::vec3 &p1, glm::vec3 &p2, glm::vec3 &p3,
float t);
	void CreatePath (glm::vec3 &p0, glm::vec3 &p1, glm::vec3 &p2, glm::vec3 &p3);
	void RenderPath();
private:
	GLuint m_vao;
};
```

You’ll see there are now two new methods: `CreatePath()`, and `RenderPath()`. In CatmullRom.cpp, implement the `CreatePath()` method. This can be achieved by copying and pasting the following code:

```
void CCatmullRom::CreatePath(glm::vec3 &p0, glm::vec3 &p1, glm::vec3 &p2, glm::vec3 &p3)
{
	// Use VAO to store state associated with vertices
	glGenVertexArrays(1, &m_vao);
	glBindVertexArray(m_vao);
	// Create a VBO
	CVertexBufferObject vbo;
	vbo.Create();
	vbo.Bind();
	glm::vec2 texCoord(0.0f, 0.0f);
	glm::vec3 normal(0.0f, 1.0f, 0.0f);
	for (unsigned int i = 0; i < 100; i++) {
		float t = (float)i / 100.0f;
		glm::vec3 v = Interpolate(p0, p1, p2, p3, t);
		vbo.AddData(&v, sizeof(glm::vec3));
		vbo.AddData(&texCoord, sizeof(glm::vec2));
		vbo.AddData(&normal, sizeof(glm::vec3));
	}
	// Upload the VBO to the GPU
	vbo.UploadDataToGPU(GL_STATIC_DRAW);

	// Set the vertex attribute locations
	GLsizei stride = 2*sizeof(glm::vec3)+sizeof(glm::vec2);

	// Vertex positions
	glEnableVertexAttribArray(0);
	glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, stride, 0);
	// Texture coordinates
	glEnableVertexAttribArray(1);
	glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, stride, (void*)sizeof(glm::vec3));
	// Normal vectors
	glEnableVertexAttribArray(2);
	glVertexAttribPointer(2, 3, GL_FLOAT, GL_FALSE, stride, (void*)(sizeof(glm::vec3)
	+sizeof(glm::vec2)));
}
```

Take a moment to appreciate what the code is doing. It creates a VAO, and binds it. It also creates a VBO, and binds that, using a helper class,

```
	CVertexBufferObject vbo;
```

This VBO is a container for 100 vertices, which are defined using attributes like position (also texture coordinate and normal). The VBO data is then sent to the GPU for later rendering. Finally, we set vertex attribute positions so that the vertex shader knows how to find the vertex attributes in the array.

See if you can write the CCatmullRom::RenderPath() method yourself. To do this, bind the VAO and call glDrawArrays using the GL_LINE_STRIP primitive with 100 vertices. You may wish to increase the line width using a call to `glLineWidth()`. (Note some drivers do not implement the `glLineWidth()` call).

Once this is implemented, instantiate a member of this class in your Game class. The best way to do this is as follows:

1. In Game.h, declare the class CCatmullRom near the top of the file, along with the other classes defined there, i.e.,

```
class CCatmullRom;
```

2. In Game.h, in the Game class definition, add a new member variable that is a pointer to a CCatmullRom datatype, i.e.
```
CCatmullRom *m_pCatmullRom;
```
3. In Game.cpp, include the header to the CatmullRom class if it is not included already, i.e.
```
#include "CatmullRom.h"
```
4. In Game.cpp, in the game constructor, Game::Game(), initialise the point to this object to NULL, i.e.
```
m_pCatmullRom = NULL;
```
5. In Game.cpp, in the game destructor, Game::~Game(), delete the pointer to this object.
```
delete m_pCatmullRom;
```
6. In Game.cpp, in Game::Initialise(), allocate memory for a new CCatmullRom object, i.e.
```
m_pCatmullRom = new CCatmullRom;
```

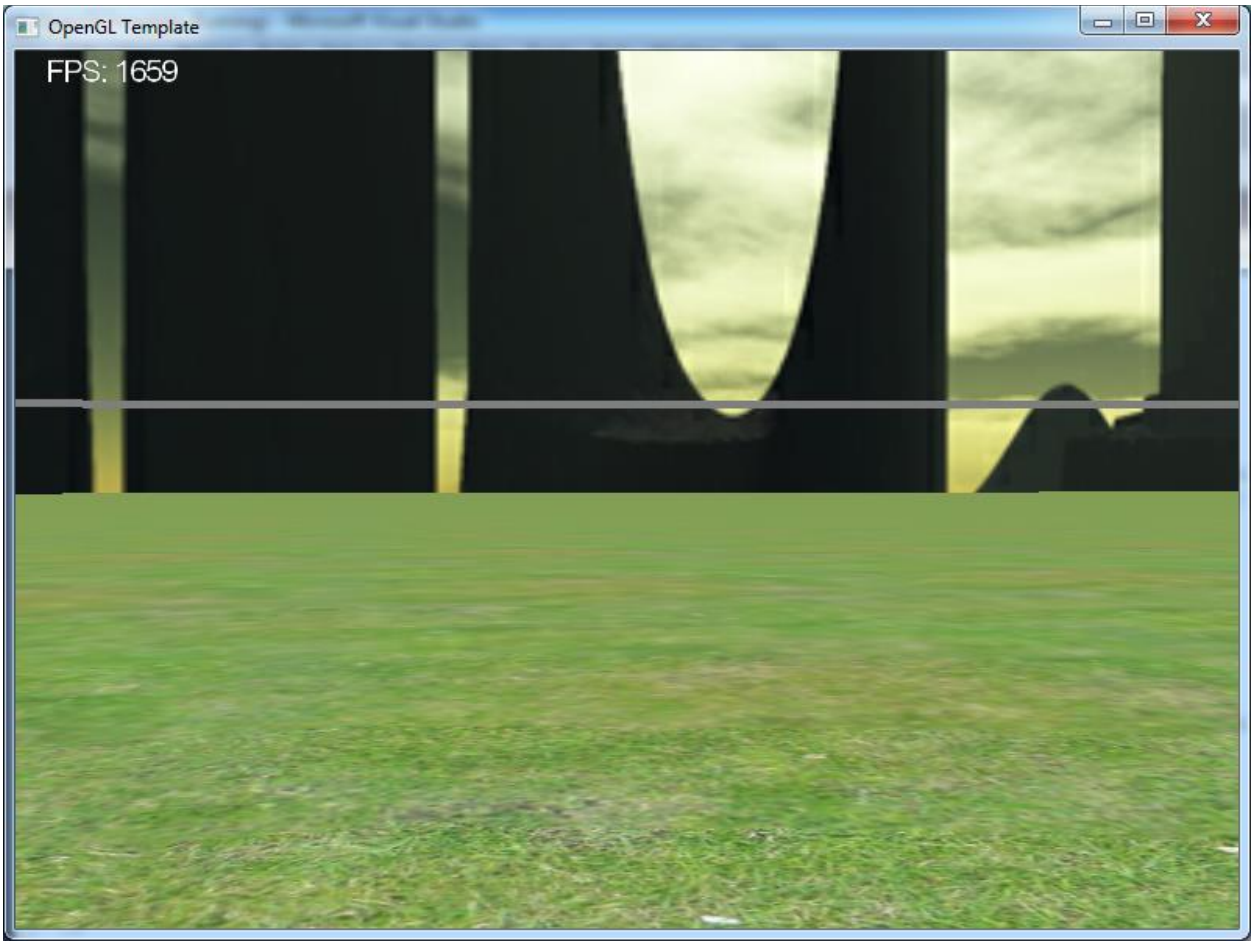
You should now have a pointer to a CCatmullRom object ready for use.

Call the `CreatePath()` method from `Game::Initialise()`, using the four points from before. Then in `Game::Render()`, call the RenderPath() method to draw the spline.

Before rendering the spline, be sure to send the modelview matrix and normal matrix to the shaders, like this:

```
modelViewMatrixStack.Push();
	pMainProgram->SetUniform("bUseTexture", false); // turn off texturing
	pMainProgram->SetUniform("matrices.modelViewMatrix", modelViewMatrixStack.Top());
	pMainProgram->SetUniform("matrices.normalMatrix",
		m_pCamera->ComputeNormalMatrix(modelViewMatrixStack.Top()));
	// Render your object here
modelViewMatrixStack.Pop();
```

We'll cover the normal matrix when we discuss lighting. Run your program. Hopefully you see the spline rendered in the game world, as below (here I set the line width to 5.0f).


OpenGL Template
FPS: 1659